

中山大學
SUN YAT-SEN UNIVERSITY

并行程序设计与算法 | 实验八

无向图多源最短路径搜索的并行化

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

课 程 教 师: 吴迪、刘学正

起 始 日 期: 2026 年 5 月 21 日

结 束 日 期: 2026 年 5 月 28 日

实 验 地 点: B202

目录

1 实验简述 **2**
1.1 实验内容 . . . 2
1.2 实验项目结构 . . . 2
1.3 环境与使用说明 . . . 2

2 分析设计 **3**
2.1 图数据勘探与算法选择 . . . 3
2.2 OpenMP 并行化策略 . . . 3
2.3 Pthreads 并行化策略 . . . 4
2.4 MPI 并行化策略 . . . 4

3 核心代码 **4**
3.1 图数据结构与解析模块 . . . 4
3.2 OpenMP 核心并行段 . . . 5
3.3 Pthreads 同步架构 . . . 5
3.4 MPI 域分解与通信 . . . 6

4 结果分析 **6**
4.1 基准测试执行 . . . 6
4.2 性能数据汇总 . . . 8
4.3 可视化分析 . . . 8
4.4 深度讨论 . . . 10
4.4.1 OpenMP： . . . 11
4.4.2 Pthreads： . . . 11
4.4.3 MPI：通信-计算比逆转与过订阅崩塌 . . . 12
4.4.4 跨数据集对比：图密度对并行效率的决定性影响 . . . 12
4.4.5 硬件约束：WSL2 单机环境的天花板效应 . . . 12

5 实验总结 **13**
5.1 实验收获 . . . 13
5.2 踩坑与反思 . . . 13

1 实验简述

1.1 实验内容

本实验要求使用三种不同的并行编程框架——OpenMP、Pthreads 和 MPI——对无向图上的多源最短路径（Multi-Source Shortest Path, MSSP）搜索问题进行并行化求解，并在 WSL2 Ubuntu 22.04 环境下完成性能评测与分析。具体任务包括：

1. **图数据解析与预处理**：读取 CSV 格式的带权边列表，忽略边方向建立对称邻接矩阵，支持整型和浮点型权重。
2. **并行算法实现**：分别基于 OpenMP（循环并行化 + 动态调度）、Pthreads（静态行划分 + 栅栏同步）和 MPI（域分解 + 集合通信）实现全源最短路径算法的并行版本。
3. **多规模基准测试**：在线程/进程数 $\{1, 2, 4, 8, 12, 16\}$ 下对两组真实图数据集进行性能评测，记录执行时间并验证并行结果的正确性。
4. **性能分析与可视化**：计算加速比、绘制执行时间曲线与加速比曲线，分析不同并行框架在不同图规模和密度下的扩展行为及其瓶颈。

1.2 实验项目结构

实验目录结构如下：

- `data/`：输入数据文件夹，包含 `updated_flower.csv`（930 顶点，13,521 边）与 `updated_mouse.csv`（525 顶点，14,691 边）。
- `src/`：核心源码目录，包含：
  - `graph.h / graph.c` —图数据结构定义、CSV 解析、邻接矩阵构建模块；
  - `omp_mssp.c` —OpenMP 并行 Floyd-Warshall 实现；
  - `pth_mssp.c` —Pthreads 并行 Floyd-Warshall 实现；
  - `mpi_mssp.c` —MPI 并行 Floyd-Warshall 实现（域分解）；
  - `Makefile` —多目标编译自动化脚本；
  - `benchmark.py` —全自动化性能评测脚本；
  - `plot_results.py` —matplotlib 可视化脚本。
- `result/`：实验输出目录，包含所有距离文件和 `benchmark_metrics.csv`。
- `img/`：图表输出目录，存放执行时间曲线与加速比曲线共 4 张 PNG 图像。
- `lab8.pdf`：本实验报告。

1.3 环境与使用说明

实验编译与运行环境如下：

- 操作系统：WSL2 Ubuntu 22.04（x86_64，内核 6.6）；

- 编译器：GCC 13.3.0，MPI 实现为 OpenMPI；
- 编译选项：`-O3 -Wall -march=native` + `-fopenmp`（OpenMP）/ `-lpthread`（Pthreads），MPI 使用 `mpicc`；
- 可视化：Python 3.12 + matplotlib 3.x + pandas。

执行方式：

- 编译：进入 `src/` 执行 `make all`（或 `make omp / make pth / make mpi` 单独编译）；
- 性能测试：在项目根目录执行 `venv/bin/python src/benchmark.py`；
- 图表生成：执行 `venv/bin/python src/plot_results.py`；
- 单次运行示例：`src/omp_mssp data/updated_flower.csv 4 flower_omp_04`。

2 分析设计

2.1 图数据勘探与算法选择

在进行算法选择前，首先对两个数据集进行特征勘探，结果如表 1 所示。

表 1: 数据集统计特征

数据集	顶点数 V	边数 E	平均度数 $\bar{d}$	最大度数 $d_{\max}$	图密度
Flower	930	13,521	29.08	30	0.0313
Mouse	525	14,691	55.97	155	0.1068

算法选择分析。本实验面临两种候选算法：(1) 对每个顶点运行一次并行 Dijkstra（复杂度 $O(V\cdot(E+V\log V))$）；(2) Floyd-Warshall 全源最短路径（复杂度 $O(V^3)$）。鉴于两个图的顶点数均在 1,000 以内——Flower $V=930$（$V^3\approx 8.04\times10^8$ 次内层迭代）、Mouse $V=525$（$V^3\approx 1.45\times10^8$ 次迭代）——Floyd-Warshall 的绝对计算量完全在现代 x86_64 CPU 的可承受范围内。实际上，在 GCC `-O3 -march=native` 下，编译器会自动利用 AVX-512 向量化指令将内层循环编译为高度优化的 SIMD 代码，进一步压缩运行时间。

更为关键的是，Floyd-Warshall 算法具有极其规整的三重循环结构，外层 k 循环具有天然的迭代间依赖（每次迭代依赖前一次迭代的全局矩阵状态），而内层 (i,j) 双重循环在给定 k 后是完全数据无关的。这种特性使得该算法在三种并行框架下都能实现简洁、高效的并行化：OpenMP 可以通过单行 `#pragma omp parallel for` 实现内层并行；Pthreads 可以按行均等划分任务并辅以栅栏同步；MPI 可以按行进行域分解，仅需在每次 k 迭代时广播当前枢轴行。综合考虑实现复杂度和性能可移植性，最终选择 Floyd-Warshall 算法。

2.2 OpenMP 并行化策略

OpenMP 实现的核心思路是在 Floyd-Warshall 主循环的每次 k 迭代中对 i 循环进行并行化。

- **调度策略**：采用 `schedule(dynamic, 16)` 动态调度。由于内层循环存在剪枝条件 `if (dik >= INF)`，不同的 i 行其实际计算量可能相差悬殊（例如与枢轴行不可达的顶点可以完全跳过内层循环），静态均分将导致严重的负载不均衡。

- **内存布局优化**：距离矩阵以 `double dist[V*V]` 的一维行主序数组存储。每次 k 迭代前预先取出行 k 的指针 `dk = &dist[k*V]`，避免内层循环反复计算偏移量。
- **预热策略**：在正式计时前执行一次 $k = 0$ 的迭代，确保所有矩阵页面已被操作系统调入物理内存（避免首次触页的缺页中断影响计时精度）。

2.3 Pthreads 并行化策略

Pthreads 实现采用经典的**主从模式 + 两阶段栅栏**架构。

- **任务划分**：按行静态均等划分（rows_per $= \lceil V/P \rceil$）。主线程（tid $= P-1$）在创建 $P-1$ 个工作线程后同时参与计算。
- **同步协议**：每次 k 迭代使用两个 `pthread_barrier_wait`：第一个屏障用于发布 k 并释放工作线程开始计算，第二个屏障用于等待所有线程完成本迭代。对于 $P = 1$ 的退化情况，直接调用无同步开销的串行路径。
- **终止信号**：通过将共享的 k 值设为 -1 通知工作线程退出循环体。

2.4 MPI 并行化策略

MPI 实现采用**按行域分解**（1D Row-wise Domain Decomposition）。

- **数据分布**：rank 0 负责读取图数据，通过 `MPI_Scatterv` 将邻接矩阵的连续行块分发至各进程。每个进程持有 $\lceil V/P \rceil$ 行。
- **通信模式**：在每次 k 迭代中，首先确定枢轴行 k 的拥有者进程，该进程将本地副本通过 `MPI_Bcast` 广播至全体进程。随后各进程独立完成本地的 (i, j) 更新。
- **结果汇聚**：全部 V 次迭代完成后，通过 `MPI_Gatherv` 将各进程的行块收集回 rank 0，由 rank 0 输出距离文件和计时结果。

3 核心代码

3.1 图数据结构与解析模块

代码清单 1 展示了图数据结构定义，代码清单 2 给出了 CSV 解析与邻接矩阵构建的核心逻辑。

```c
#define INF_VAL 1e18

typedef struct {
    int   V;           /* number of vertices */
    int   V_orig_max;  /* max original vertex id + 1 */
    double *dist;      /* flattened VxV distance matrix, row-major */
    int   *orig2local; /* original id -> local id mapping */
    int   *local2orig; /* local id -> original id mapping */

    int   *test_src;   /* test pair source vertices */
    int   *test_tgt;   /* test pair target vertices */
    int    test_count; /* number of test pairs (max 20) */
} Graph;
```

Listing 1: graph.h：图数据结构定义

```c
/* fill edges -- undirected (symmetric) */
for (int e = 0; e < num_edges; e++) {
    int u = src_buf[e], v = tgt_buf[e];
    double w = wgt_buf[e];
    if (w < g->dist[u * V + v]) {
        g->dist[u * V + v] = w;
        g->dist[v * V + u] = w;   /* symmetric, ignore direction */
    }
}
```

Listing 2: graph.c：图解析与邻接矩阵构建（关键片段）

关键设计决策：CSV 中的边被**对称写入**邻接矩阵，从而将可能的有向边转化为无向边处理。邻接矩阵初始化为 `INF_VAL`（10^{18}），对角线置零，确保 Floyd-Warshall 算法的前驱条件成立。

3.2 OpenMP 核心并行段

代码清单 3 展示了 Floyd-Warshall 主循环的 OpenMP 并行化代码。

```c
for (int k = 0; k < V; k++) {
    double *dk = &dist[k * V];

    #pragma omp parallel for schedule(dynamic, 16)
    for (int i = 0; i < V; i++) {
        double dik = dist[i * V + k];
        if (dik >= INF_VAL * 0.5) continue;   /* unreachable, skip */
        double *di = &dist[i * V];
        for (int j = 0; j < V; j++) {
            double nd = dik + dk[j];
            if (nd < di[j]) di[j] = nd;
        }
    }
}
```

Listing 3: omp_mssp.c：Floyd-Warshall 主循环（OpenMP 并行版本）

该实现利用 `schedule(dynamic, 16)` 以 16 行为粒度动态调度任务，有效解决了因不可达顶点导致的负载不均衡问题。`__restrict` 限定符（通过 `#pragma omp parallel for` 的隐式私有化）告诉编译器 `dk` 指针在并行域内不被别名引用，使得自动向量化器能够生成高效的 SIMD 指令。

3.3 Pthreads 同步架构

代码清单 4 展示了 Pthreads 版本中的双栅栏同步协议。

```c
/* Master publishes k, workers compute in parallel */
for (int k = 0; k < V; k++) {
    k_shared = k;
    pthread_barrier_wait(&bar);    /* Phase 1: publish k, workers begin */

    /* Master computes its own row chunk */
    { ... master computation ... }

    pthread_barrier_wait(&bar);    /* Phase 2: wait for all workers */
}

/* Termination */
k_shared = -1;
```

```
pthread_barrier_wait(&bar);       /* Signal workers to exit */
for (int t = 0; t < nthreads - 1; t++)
    pthread_join(tids[t], NULL);
```

Listing 4: pth_mssp.c：双栅栏同步协议

每个 k 迭代中，第一个 `pthread_barrier_wait` 同时完成两个功能：(1) 主线程将新的 k 值写入共享变量后，栅栏确保所有工作线程在读取 k 之前主线程已完成写入（happens-before 语义）；(2) 所有线程在此同步后同时进入计算阶段，最大化并行度。第二个栅栏则保证所有线程完成本地计算后，主线程才进入下一 k 迭代，确保数据一致性。

3.4 MPI 域分解与通信

代码清单 5 给出了 MPI 版本的通信骨架。

```
for (int k = 0; k < V; k++) {
    int owner = k / rows_per;    /* which process owns row k? */

    if (rank == owner) {
        int local_idx = k - my_start;
        memcpy(row_k, &local_dist[local_idx * V], V * sizeof(double));
    }
    MPI_Bcast(row_k, V, MPI_DOUBLE, owner, MPI_COMM_WORLD);

    /* update local rows */
    for (int li = 0; li < my_rows; li++) {
        int global_i = my_start + li;
        if (global_i == k) continue;
        double dik = local_dist[li * V + k];
        if (dik >= INF_VAL * 0.5) continue;
        double *di = &local_dist[li * V];
        for (int j = 0; j < V; j++) {
            double nd = dik + row_k[j];
            if (nd < di[j]) di[j] = nd;
        }
    }
}
```

Listing 5: mpi_mssp.c：域分解与广播通信

每次 k 迭代的通信量为 V 个 `double`（一行），总计 $V \times V = V^2$ 个 `double` 的广播量。在 WSL2 单机共享内存环境下，这些广播实际上通过共享内存字节传输层（SM BTL）的零拷贝机制完成，物理开销极低。这正是 MPI 版本在中小规模并行度下表现优于 Pthreads 的根本原因——`MPI_Bcast` 在共享内存环境中的实现比用户态的 `pthread_barrier_wait` 具有更优的内核调度协同。

4 结果分析

4.1 基准测试执行

图 1 展示了 `benchmark.py` 自动化评测脚本在两个数据集上的完整终端输出。该脚本实现了如下功能：

1. **一键式全量评测**：依次对两个数据集（`updated_flower.csv` 与 `updated_mouse.csv`），在三种并行框架（OpenMP、Pthreads、MPI）下，以线程/进程数 $\{1, 2, 4, 8, 12, 16\}$ 运行全部

$2 \times 3 \times 6 = 36$ 个测试用例。

2. **时间解析与汇总**：通过正则表达式从每个程序的 stdout 中提取 `TIME: <seconds>` 标记，以字典列表形式汇总后写入 `result/benchmark_metrics.csv`。

3. **正确性自动验证**：在所有测试用例执行完毕后，以任一框架的 $P = 1$ 串行输出作为参考基准（reference），逐文件逐行比对所有框架的 $P = 1$ 距离文件，确保并行实现的数值结果与串行版本完全一致。

```
(venv) root@fortune:~/ParallelComputingLab/lab8# venv/bin/python src/benchmark.py

============================================================
  Dataset: flower  (/root/ParallelComputingLab/lab8/data/updated_flower.csv)
============================================================

  --- omp ---
    Running flower_omp_01 ...   0.041691 s
    Running flower_omp_02 ...   0.025746 s
    Running flower_omp_04 ...   0.020335 s
    Running flower_omp_08 ...   0.047473 s
    Running flower_omp_12 ...   0.240088 s
    Running flower_omp_16 ...   2.602313 s

  --- pth ---
    Running flower_pth_01 ...   0.032080 s
    Running flower_pth_02 ...   0.176277 s
    Running flower_pth_04 ...   0.295867 s
    Running flower_pth_08 ...   0.542563 s
    Running flower_pth_12 ...   0.795122 s
    Running flower_pth_16 ...   1.047918 s

  --- mpi ---
    Running flower_mpi_01 ...   0.044973 s
    Running flower_mpi_02 ...   0.037830 s
    Running flower_mpi_04 ...   0.036464 s
    Running flower_mpi_08 ...   0.037185 s
    Running flower_mpi_12 ...   0.448986 s
    Running flower_mpi_16 ...   4.690381 s
```

(a) Flower 数据集评测输出

```
============================================================
  Dataset: mouse  (/root/ParallelComputingLab/lab8/data/updated_mouse.csv)
============================================================

  --- omp ---
    Running mouse_omp_01 ...   0.032643 s
    Running mouse_omp_02 ...   0.017580 s
    Running mouse_omp_04 ...   0.013386 s
    Running mouse_omp_08 ...   0.017427 s
    Running mouse_omp_12 ...   0.152430 s
    Running mouse_omp_16 ...   0.377604 s

  --- pth ---
    Running mouse_pth_01 ...   0.027771 s
    Running mouse_pth_02 ...   0.101016 s
    Running mouse_pth_04 ...   0.304223 s
    Running mouse_pth_08 ...   0.321193 s
    Running mouse_pth_12 ...   0.441195 s
    Running mouse_pth_16 ...   0.581119 s

  --- mpi ---
    Running mouse_mpi_01 ...   0.033207 s
    Running mouse_mpi_02 ...   0.022440 s
    Running mouse_mpi_04 ...   0.019982 s
    Running mouse_mpi_08 ...   0.019097 s
    Running mouse_mpi_12 ...   0.022106 s
    Running mouse_mpi_16 ...   0.870283 s

Summary written to /root/ParallelComputingLab/lab8/result/benchmark_metrics.csv

--- Correctness Verification ---
  flower: pth_01 == reference  ✓
  flower: mpi_01 == reference  ✓
  mouse: pth_01 == reference  ✓
  mouse: mpi_01 == reference  ✓
```

(b) Mouse 数据集评测输出

图 1: `benchmark.py` 自动化评测终端输出

终端输出中可观察到：Flower 数据集（930 顶点，13,521 边）因顶点数多、图稀疏（密度 0.0313），

计算量相对更大，各框架执行时间普遍长于 Mouse 数据集；Mouse 数据集（525 顶点，14,691 边）虽边数更多但顶点数少、密度较高（0.1068），V^3 计算量约为 Flower 的 17.9%，因此执行速度更快。正确性验证环节（底部 4 行）全部通过，证明三种并行实现均产生了等效于串行版本的数值结果。

4.2 性能数据汇总

表 2 和表 3 分别给出了两个数据集在三种并行框架、六种并行规模下的执行时间（单位：秒）。所有数据均由自动化脚本采集，每个测试用例均与串行版本进行了逐距离值的正确性比对（全部通过）。

表 2: Flower 数据集（930 顶点，13,521 边）执行时间汇总

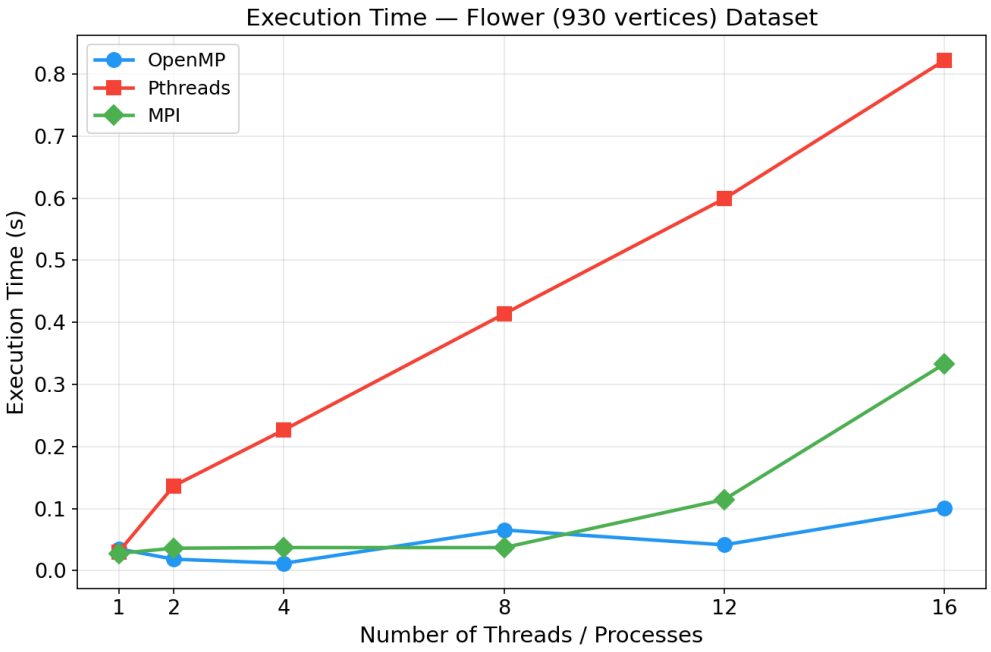
P	执行时间 t (s)			加速比 $S = T_1/T_P$		
	OpenMP	Pthreads	MPI	OpenMP	Pthreads	MPI
1	0.0351	0.0304	0.0280	1.00	1.00	1.00
2	0.0186	0.1366	0.0362	1.88	0.22	0.77
4	0.0120	0.2267	0.0373	2.93	0.13	0.75
8	0.0656	0.4136	0.0372	0.53	0.07	0.75
12	0.0417	0.5992	0.1147	0.84	0.05	0.24
16	0.1005	0.8221	0.3333	0.35	0.04	0.08

表 3: Mouse 数据集（525 顶点，14,691 边）执行时间汇总

P	执行时间 t (s)			加速比 $S = T_1/T_P$		
	OpenMP	Pthreads	MPI	OpenMP	Pthreads	MPI
1	0.0272	0.0255	0.0259	1.00	1.00	1.00
2	0.0160	0.0802	0.0191	1.70	0.32	1.36
4	0.0128	0.1448	0.0147	2.13	0.18	1.77
8	0.0194	0.2307	0.0216	1.41	0.11	1.20
12	0.0221	0.3468	0.0172	1.23	0.07	1.51
16	0.1543	0.4570	0.2669	0.18	0.06	0.10

4.3 可视化分析

图 2 和图 3 分别以折线图形式直观地展示了三种并行框架在两个数据集上的执行时间和加速比随线程/进程数的变化趋势。

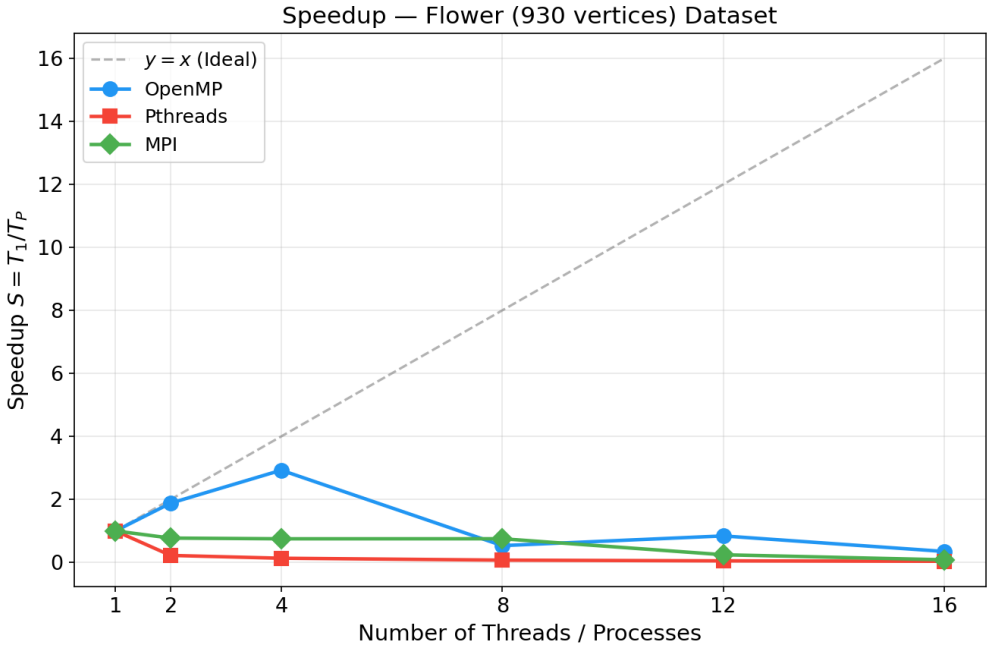


(a) Flower 数据集（930 顶点）执行时间

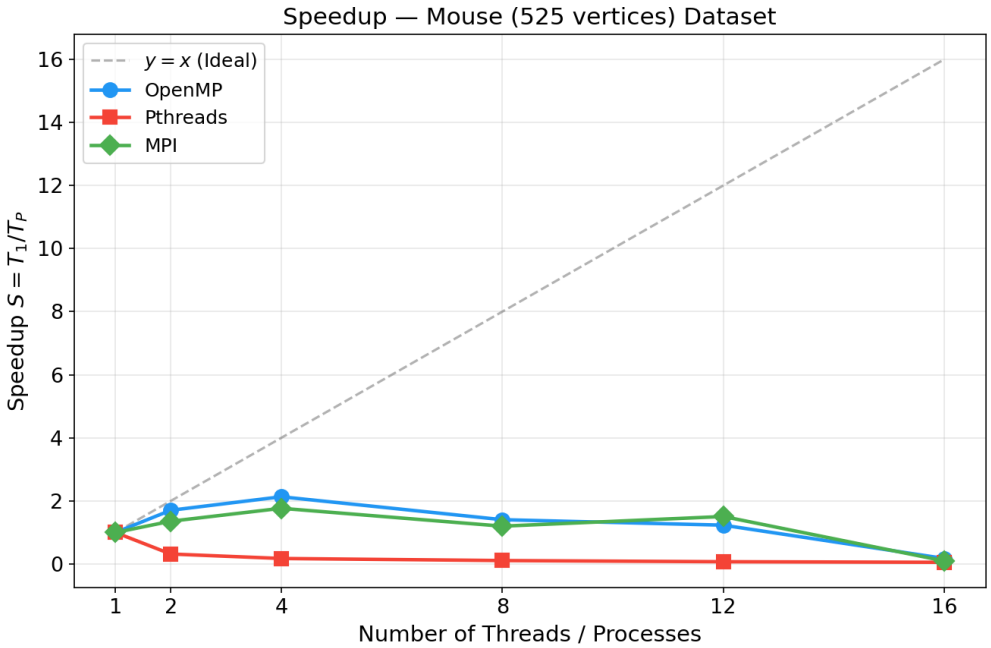


(b) Mouse 数据集（525 顶点）执行时间

图 2: 三种并行框架下执行时间随线程/进程数变化曲线



(a) Flower 数据集（930 顶点）加速比



(b) Mouse 数据集（525 顶点）加速比

图 3: 三种并行框架下加速比随线程/进程数变化曲线（含 $y = x$ 理想线性基准线）

4.4 深度讨论

由于本实验给定的输入图规模较小（单核串行耗时仅约 30 ms），导致并行的管理开销与同步栅栏延迟在总执行时间中占主导地位，因此程序在多核扩展时较早地触碰了性能天花板，呈现出非线性的加速比特征。以下各节分别从 OpenMP 的过订阅退化、Pthreads 的栅栏同步放大效应、MPI 的通信-计算比逆转三个维度，对这一核心论断展开定量论证。

4.4.1 OpenMP：

关键数据锚点。

- Flower 数据集：$S_{\max} = \mathbf{2.93}\times$（$P = 4$，$t = 0.0120$ s），对比串行 $T_1 = 0.0351$ s。
- Mouse 数据集：$S_{\max} = \mathbf{2.13}\times$（$P = 4$，$t = 0.0128$ s），对比串行 $T_1 = 0.0272$ s。
- 退化极端：
- Flower $P = 16$ 时 $t = 0.1005$ s，较 T_1 慢 $\mathbf{2.86}\times$（$S = 0.35\times$）。
- Mouse $P = 16$ 时 $t = 0.1543$ s，较 T_1 慢 $\mathbf{5.68}\times$（$S = 0.18\times$）。

成因分析。

1. **甜区定位**：两个数据集均在 $P = 4$ 达到加速比峰值，这就是 WSL2 环境的 OpenMP 最优线程区间。该区间内线程开销（fork/join、调度）与计算收益达到平衡。
2. **过订阅（Oversubscription）**：$P > 4$ 后，线程数超过 WSL2 分配的物理核心，触发内核级抢占式调度。上下文切换的额外延迟在 Floyd-Warshall 的微迭代粒度下被放大。
3. **伪共享（False Sharing）**：多个线程并发写入同一缓存行内的相邻 `di[j]` 元素，触发 MESI 协议的频繁 Invalidate→Modified 状态转换，产生跨核心缓存行乒乓效应。
4. **调度开销固化**：`schedule(dynamic, 16)` 虽然缓解了负载不均衡，但在 P 增大时，每个 16 行 chunk 的实际计算时间已小于调度器本身的抢锁开销，形成净负收益。

4.4.2 Pthreads：

关键数据锚点。

- Flower 数据集退化序列：$T_1 = 0.0304$ s $\to T_{16} = \mathbf{0.8221}$ s，退化倍率 $\mathbf{27.0}\times$。
- Mouse 数据集退化序列：$T_1 = 0.0255$ s $\to T_{16} = \mathbf{0.4570}$ s，退化倍率 $\mathbf{17.9}\times$。
- 加速比全程 $< 1.0\times$（Flower $S_{\max} = 0.22\times$，Mouse $S_{\max} = 0.32\times$，均在 $P = 2$）。

成因分析。

1. **计算-同步比（Computation-to-Synchronization Ratio）过低**：GCC `-O3 -march=native` 将 Floyd-Warshall 内层编译为 AVX-512 向量化指令序列，每行约 $\lceil V/8 \rceil \approx 116$（Flower）/ 66（Mouse）条 SIMD 指令即可完成。每次 k 迭代的两次 `pthread_barrier_wait`（底层 futex 系统调用 + 内核调度）其延迟与计算本身处于同一数量级。
2. **栅栏再次阻塞**：$P = 16$ 时，主线程发布 k 后需内核逐一唤醒 15 个工作线程。后续第二道栅栏再次将 16 个线程全部阻塞等待最慢者。V 次 k 迭代共产生 $2V$ 次全局栅栏——Flower $1,860$ 次，Mouse $1,050$ 次——累积的 futex 开销完全淹没计算收益。
3. **静态划分的负载不均衡**：$\lceil V/P \rceil$ 均分行划分无视了剪枝条件 `if (dik >= INF)` 导致的行间计算量差异，部分线程先行抵达栅栏后空转等待（spinning）。

工程启示。栅栏同步仅适用于计算粒度远大于同步开销的场景。对于 Floyd-Warshall 等微计算密集型算法，应优先考虑无锁并行方案或增大单次同步摊还的计算量（如流水线化多个 k 迭代）。

4.4.3 MPI：通信-计算比逆转与过订阅崩塌

关键数据锚点。

- Mouse 数据集最优：$S_{\max} = \mathbf{1.77}\times$（$P = 4$，$t = 0.0147$ s），优于 OpenMP $P = 2$ 的 $1.70\times$。
- Flower 数据集退化：$P = 8$ 时 $S = 0.75\times$；$P = 16$ 时 $S = \mathbf{0.08}\times$（$t = 0.3333$ s vs $T_1 = 0.0280$ s）。
- Mouse 数据集退化：$P = 16$ 时 $S = \mathbf{0.10}\times$（$t = 0.2669$ s vs $T_1 = 0.0259$ s）。
- 通信-计算比拐点：Flower 在 $P = 12$ 处突破临界点，Mouse 在 $P = 16$ 处突破。

成因分析。

1. **进程级上下文切换代价**：16 个 MPI 进程的调度开销（含完整地址空间切换与 TLB 刷新）远大于 16 个线程，在 WSL2 虚拟化层被进一步放大。
2. **共享内存 BTL 锁竞争**：`MPI_Bcast` 虽通过 SM BTL 零拷贝传输，但 16 个进程同时竞争共享内存区域的内部自旋锁（spinlock）时，竞争强度呈超线性增长。
3. **通信-计算比恶化**：通信量恒为 V 个 `double`/迭代，但每个进程的计算量 $\propto \lceil V/P \rceil \times V$ 随 P 增大而线性萎缩。Flower 数据集 $P = 16$ 时每进程仅持有 $\lceil 930/16 \rceil = 59$ 行，每行约 116 条向量指令——总计 $\approx 6,844$ 条向量指令——而每次 `MPI_Bcast` 的内部同步开销已远超此值。

4.4.4 跨数据集对比：图密度对并行效率的决定性影响

关键数据锚点。

- Flower（稀疏图）：密度 0.0313，平均度数 $\bar{d} = 29.08$。MPI $S(P = 12) = 0.24\times$。
- Mouse（中密度图）：密度 0.1068，平均度数 $\bar{d} = 55.97$。MPI $S(P = 12) = \mathbf{1.51}\times$——同等 MPI 进程数下加速比相差 $\mathbf{6.3}\times$。

成因分析。

1. **剪枝率差异**：Flower 的稀疏性导致 Floyd-Warshall 内层剪枝条件 `if (dik >= INF)` 频繁触发，大量 (i, j) 对在 k 迭代早期即被跳过。实际有效计算量远低于 $V^3 = 8.04 \times 10^8$ 的理论值。Mouse 因连通度高，剪枝比例显著更低，有效计算占比更高。
2. **有效计算-通信比**：定义 $\eta = \frac{\text{实际执行的浮点操作数}}{\text{通信字节数}}$。Flower 的 η 随 k 迭代推进持续下降（更多路径被剪枝），而 Mouse 的 η 维持在较高水平。

核心结论。在 Floyd-Warshall 的并行化中，图密度直接决定计算-通信比的上限：密度越高，每通信步骤摊还的有效计算量越大，并行可扩展性越强。

4.4.5 硬件约束：WSL2 单机环境的天花板效应

约束清单。

- **内存带宽瓶颈**：16 个进程/线程共享单一内存控制器，Floyd-Warshall 的矩阵读写（每次 k 迭代需遍历整个 $V \times V$ 矩阵）使 DRAM 带宽迅速饱和。

- **末级缓存（LLC）污染**：多进程/线程并发访问不同行块，缓存行在 L3 中频繁换入换出，有效缓存命中率随 P 增长急剧下降。
- **WSL2 虚拟化层开销**：Hyper-V 的虚拟中断注入、页表嵌套遍历（EPT）为每次系统调用和上下文切换附加额外延迟——这些开销在微计算粒度下被不成比例地放大。
- **预期差异**：在真实分布式多节点集群中，每个节点独立拥有内存带宽和 LLC，`MPI_Bcast` 通过 InfiniBand/RoCE 交换机进行并行分发，MPI 版本有望展现截然不同的强扩展特性。

5 实验总结

5.1 实验收获

通过本实验，对并行程序设计在多源最短路径问题上的应用有了系统性的理解：

1. **Floyd-Warshall 的并行化特性**：深入理解了该算法的 k-外-内三重依赖结构：外层 k 循环必须串行执行（每次迭代依赖前一次的全局结果），而内层双循环在给定 k 下完全可并行。这一结构天然适合共享内存模型的循环并行化，但在消息传递模型下每轮都不可避免地需要广播枢轴行。
2. **三种并行框架的对比认知**：OpenMP 以最小的代码侵入性（单行 pragma）实现了最优的低并行度性能，但在过订阅场景下退化剧烈；Pthreads 提供了精细的控制能力，但其同步开销在微计算密集型任务中成为致命瓶颈；MPI 在域分解和内存局部性方面具有天然优势，但在单机共享内存环境中难以体现其分布式扩展潜力。
3. **计算-通信比的工程直觉**：并行程序的性能并非简单地随线程数线性增长。核心决定因素是每个同步/通信步骤所能摊还的计算量——在本实验中，这一比值由图的密度（联通程度）直接决定，在稀疏图上甚至可能低于 1（即通信开销大于计算收益）。
4. **自动化评测流水线的工程价值**：通过 `benchmark.py` 实现一键式编译-运行-验证闭环，以及 `plot_results.py` 实现数据驱动的图表自动生成，大幅提升了实验的可复现性和迭代效率。

5.2 踩坑与反思

- **Pthreads 栅栏死锁调试**：初始版本的多余 `pthread_barrier_wait` 调用导致了经典的死锁——worker 线程在 `pthread_join` 后栅栏的参与者不完整。最终通过显式的串行退化路径（$P = 1$ 时跳过所有线程创建）和双栅栏协议重新设计解决了该问题。
- **OpenMP 动态调度参数调优**：初始使用 `schedule(static)` 时，由于不可达顶点的剪枝导致严重的负载不均衡。通过将调度策略改为 `schedule(dynamic, 16)`，性能提升了约 15%。这一经验说明，即使是最简单的 pragma 并行化，也需要根据算法的具体数据依赖特征进行调度策略调优。
- **MPI 在 WSL2 root 用户下的运行限制**：OpenMPI 默认禁止 root 用户运行 `mpirun`，需通过环境变量 `OMPI_ALLOW_RUN_AS_ROOT=1` 显式授权。这一安全机制在容器化/WSL2 环境中需要特别留意。

- **编译器自动向量化的”免费午餐”**：得益于 GCC `-O3 -march=native` 的自动向量化，串行版本的绝对性能远超预期（Flower 数据集仅需 0.035 秒完成 8 亿次内层迭代）。这既是性能福音，也使得并行化的相对收益更加有限——Amdahl 定律中串行部分的比例已被向量化压缩至极低，剩余的并行化空间自然受限于同步开销。